

EliteRestaurant - Friendly Project Guide

This document explains the whole project in human language, mixed with practical software terms, so you can quickly understand what is connected to what, where to look in code, and how to navigate both the app and repository.

Generated: 2026-04-02 04:47

Scanned source files: 70 (52 C#, 17 XAML, 1 project files), total 11,741 lines.

How to read this guide

- Think of the app as three layers: UI (Views), screen logic (ViewModels), and data engine (Data + Models).
- When a user clicks a button, a command in a ViewModel runs business logic and usually reads/writes SQLite through AppDbContext.
- Every section tells you where to click in the app and where the equivalent code lives.
- This is an onboarding map: if you are lost, follow the navigation flow section first.

1) Big picture: what this project is

- This is a single desktop app called EliteRestaurantPro (not a web backend + frontend split).
 - Framework stack: .NET 8 + WPF for UI + MVVM architecture.
 - Storage stack: EF Core with SQLite local database (file in LocalApplicationData).
 - Reporting stack: ClosedXML for Excel exports and QuestPDF for PDF tickets/documents.
 - Startup bootstraps database, repairs missing schema fields, seeds data, and opens the main window.
- Core boot files: App.xaml, App.xaml.cs, MainWindow.xaml, MainWindow.xaml.cs, MainViewModel.cs.

2) Project structure in plain English

- **Data/**: the database brain. AppDbContext defines tables, relations, schema fixes, and auto-seeding.
FinancialTransactionService keeps revenue/salary ledger in sync.
- **Models/**: pure data contracts (Employee, Product, Table, OrderRecord, etc.). Think: shape of stored data.
- **ViewModels/**: feature controllers. Each screen has one ViewModel with commands, validation, and business rules.
- **Views/**: XAML UI screens. They bind to ViewModels and trigger commands. Minimal logic in code-behind.
- **Resources/**: shared visual style system (colors, controls, app theme).
- **Utils/**: helper services, for example unique ID generation and Excel export.

Fast mental model: View = what user sees, ViewModel = what user actions do, Model = what data means, Data = where data persists.

3) How everything is connected

Connection chain for nearly every feature: XAML Button -> ICommand (RelayCommand) -> ViewModel method -> ApplicationDbContext query/update -> SaveChanges -> UI refresh.

- App startup (`App.OnStartup`) initializes database first, then shows `MainWindow`.
- Main window hosts a `ContentControl` bound to `MainViewModel.CurrentViewModel`.
- App.xaml DataTemplates auto-map each ViewModel class to its View class.
- AdminBaseViewModel centralizes side-menu navigation commands for all admin pages.
- All major screens instantiate `AppDbContext` per operation (`using var db = new AppDbContext();`).

Important coupling points

- Orders module is tightly connected to Inventory (ingredient consumption) and Money (auto revenue posting).
- Attendance is connected to Money through salary scheduling logic.
- Dashboard and Reports consume data from most entities, so they act as read-heavy aggregation layers.
- Table status depends on order status transitions, not just manual table edits.

4) How to navigate the APP (user journey)

Login to admin area

Role Selection -> Admin Login -> Admin Dashboard. Current login is demo mode (non-empty credentials).

Sidebar navigation map

- Dashboard: quick KPIs, chart, and latest activities.
- Employees: add/edit/remove staff, roles, hourly rate, contact and shifts.
- Menu: manage products, categories/subcategories, and prices.
- Inventory: stock quantities, units, expiration metadata, and adjustments.
- Attendance: daily clock-in/clock-out and late justification handling.
- Tables: dining table configuration, capacity, status, assigned server.
- Orders: list/manage active and past orders with state transitions.
- Create Order: compose order items, validate stock, calculate totals, save.
- Money: revenues/expenses and financial report/export.
- Reports: deeper report views and Excel exports by entity and date windows.

Navigation in CODE

- Main routing lives in MainViewModel + DataTemplate mappings in App.xaml.
- AdminBaseViewModel builds ICommand navigation endpoints for every admin screen.
- Each ViewModel exposes page-specific commands bound in its XAML view.

5) Database lifecycle and data model

AppDbContext acts like an all-in-one persistence engine: EF DbContext + schema safety net + seeder. When app starts, it ensures DB exists, patches missing columns/tables, enforces unique indexes, and fills baseline data so the app always opens with usable content.

- Core entities: Employee, Product, Table, OrderRecord, OrderItem, InventoryItem, ProductIngredient, EmployeeAttendance, MoneyTransaction.
- Relationship examples: OrderRecord has many OrderItems; OrderItem points to Product; ProductIngredient links Product and InventoryItem.
- Business constraints: unique IDs, unique table number, unique attendance per employee/day.
- Normalization pass fixes data quality (missing IDs, duplicate IDs, empty names, section defaults).
- Historical simulation seeds realistic attendance/orders so dashboards and reports have data immediately.

Data integrity strategy

- Generate prefixed IDs (`EMP`, `TBL`, `MEN`, `INV`, `ORD`) for stable references across modules.
- Detect and repair duplicate unique IDs before creating unique indexes.
- Rebuild old `Orders` table schema when critical columns are missing or incompatible.
- Ensure money ledger has derived entries (completed-order revenue and scheduled salary expenses).

6) Core business logic explained

Orders + Inventory + Money triangle

This is the most important subsystem. Creating/completing orders does not only touch order tables; it also updates stock and can auto-create financial transactions. In other words, one user action propagates across three domains.

- CreateOrderViewModel and AdminOrdersViewModel build line items and compute subtotal/tax/service totals.
- Before commit, required ingredient quantities are computed from ProductIngredient x item quantity.
- If any inventory item is insufficient, order creation is blocked with validation feedback.
- On success, stock is decremented and adjustment notes are persisted for traceability.
- On completion, FinancialTransactionService posts a single Revenue/Sale transaction if missing.
- Dedup key for auto revenue is deterministic (`Justification` built from order reference).

Attendance + salary pipeline

- AttendanceViewModel records clock-in/clock-out for today, with late-status handling.
- Money module calls EnsureScheduledSalaryExpenses over date ranges.
- Each active employee gets Friday expense entries computed as hourly rate x 40.
- Dedup is enforced by deterministic justification string, so repeated syncs stay idempotent.

Other practical algorithms

- Prep-time estimator combines item prep heuristics and partial parallel cooking assumptions.
- Dashboard chart builder aggregates daily values into visual datapoints.
- Tables auto-switch between Available/Occupied depending on active order states.

7) Developer navigation guide (where to edit what)

- Want to change app routing/start page? Edit `MainViewModel` and `App.xaml` DataTemplates.
- Want to change visual layout/style? Edit `Views/\*.xaml` and `Resources/Styles.xaml`.
- Want to change feature behavior? Edit corresponding `ViewModels/\*ViewModel.cs`.
- Want to change table schema or bootstrap data? Edit `Data/AppDbContext.cs`.
- Want to change finance auto-sync rules? Edit `Data/FinancialTransactionService.cs`.
- Want to add export format behavior? Edit `Utils/ExcelExportService.cs` and report ViewModels.

Recommended reading order for a new developer

- Start: `EliteRestaurantPro.csproj` to understand runtime stack.
- Then: `App.xaml`, `App.xaml.cs`, `MainWindow.xaml`, `MainViewModel.cs`.
- Then: `Data/AppDbContext.cs` to understand data foundations and constraints.
- Then: feature ViewModels in this order: Dashboard -> CreateOrder -> AdminOrders -> Money -> Reports.
- Finally: Models and Views for details and UI binding verification.

8) Source index (non-generated files)

Complete list of source files scanned for this guide (.cs, .xaml, .csproj), excluding bin/obj.

EliteRestaurantPro/App.xaml (71 lines)
EliteRestaurantPro/App.xaml.cs (15 lines)
EliteRestaurantPro/AssemblyInfo.cs (10 lines)
EliteRestaurantPro/Data/AppDbContext.cs (1029 lines)
EliteRestaurantPro/Data/FinancialTransactionService.cs (116 lines)
EliteRestaurantPro/EliteRestaurantPro.csproj (20 lines)
EliteRestaurantPro/MainWindow.xaml (72 lines)
EliteRestaurantPro/MainWindow.xaml.cs (38 lines)
EliteRestaurantPro/Models/ActivityItem.cs (8 lines)
EliteRestaurantPro/Models/Employee.cs (89 lines)
EliteRestaurantPro/Models/EmployeeAttendance.cs (13 lines)
EliteRestaurantPro/Models/InventoryItem.cs (44 lines)
EliteRestaurantPro/Models/MoneyTransaction.cs (12 lines)
EliteRestaurantPro/Models/OrderEntry.cs (29 lines)
EliteRestaurantPro/Models/OrderItem.cs (11 lines)
EliteRestaurantPro/Models/OrderRecord.cs (19 lines)
EliteRestaurantPro/Models/Product.cs (12 lines)
EliteRestaurantPro/Models/ProductIngredient.cs (11 lines)
EliteRestaurantPro/Models/ReservationEntry.cs (28 lines)
EliteRestaurantPro/Models/Table.cs (13 lines)
EliteRestaurantPro/Resources/Styles.xaml (710 lines)
EliteRestaurantPro/Utils/ExcelExportService.cs (55 lines)
EliteRestaurantPro/Utils/UniqueIdGenerator.cs (10 lines)
EliteRestaurantPro/ViewModels/AdminBaseViewModel.cs (40 lines)
EliteRestaurantPro/ViewModels/AdminDashboardViewModel.cs (374 lines)
EliteRestaurantPro/ViewModels/AdminLoginViewModel.cs (61 lines)
EliteRestaurantPro/ViewModels/AdminOrdersViewModel.cs (597 lines)
EliteRestaurantPro/ViewModels/AttendanceViewModel.cs (315 lines)
EliteRestaurantPro/ViewModels/BaseViewModel.cs (20 lines)
EliteRestaurantPro/ViewModels/CreateOrderViewModel.cs (704 lines)
EliteRestaurantPro/ViewModels/DashboardDrilldownViewModel.cs (121 lines)
EliteRestaurantPro/ViewModels/EmployeesViewModel.cs (421 lines)
EliteRestaurantPro/ViewModels/InventorySelectionItemViewModel.cs (25 lines)
EliteRestaurantPro/ViewModels/InventoryViewModel.cs (363 lines)
EliteRestaurantPro/ViewModels/MainViewModel.cs (22 lines)
EliteRestaurantPro/ViewModels/MenuViewModel.cs (467 lines)
EliteRestaurantPro/ViewModels/MoneyViewModel.cs (688 lines)
EliteRestaurantPro/ViewModels/ProductSelectionItemViewModel.cs (33 lines)
EliteRestaurantPro/ViewModels/RelayCommand.cs (40 lines)
EliteRestaurantPro/ViewModels/ReportsViewModel.cs (810 lines)
EliteRestaurantPro/ViewModels/RoleSelectionViewModel.cs (17 lines)

EliteRestaurantPro/ViewModels/TablesViewModel.cs (310 lines)
EliteRestaurantPro/Views/AdminDashboardView.xaml (406 lines)
EliteRestaurantPro/Views/AdminDashboardView.xaml.cs (8 lines)
EliteRestaurantPro/Views/AdminLoginView.xaml (99 lines)
EliteRestaurantPro/Views/AdminLoginView.xaml.cs (15 lines)
EliteRestaurantPro/Views/AdminOrdersView.xaml (387 lines)
EliteRestaurantPro/Views/AdminOrdersView.xaml.cs (8 lines)
EliteRestaurantPro/Views/AttendanceView.xaml (159 lines)
EliteRestaurantPro/Views/AttendanceView.xaml.cs (11 lines)
EliteRestaurantPro/Views/Components/AdminSidebar.xaml (297 lines)
EliteRestaurantPro/Views/Components/AdminSidebar.xaml.cs (81 lines)
EliteRestaurantPro/Views/CreateOrderView.xaml (308 lines)
EliteRestaurantPro/Views/CreateOrderView.xaml.cs (11 lines)
EliteRestaurantPro/Views/DashboardDrilldownView.xaml (76 lines)
EliteRestaurantPro/Views/DashboardDrilldownView.xaml.cs (11 lines)
EliteRestaurantPro/Views/EmployeesView.xaml (399 lines)
EliteRestaurantPro/Views/EmployeesView.xaml.cs (11 lines)
EliteRestaurantPro/Views/InventoryView.xaml (242 lines)
EliteRestaurantPro/Views/InventoryView.xaml.cs (11 lines)
EliteRestaurantPro/Views/MenuView.xaml (281 lines)
EliteRestaurantPro/Views/MenuView.xaml.cs (11 lines)
EliteRestaurantPro/Views/MoneyView.xaml (279 lines)
EliteRestaurantPro/Views/MoneyView.xaml.cs (11 lines)
EliteRestaurantPro/Views/ReportsView.xaml (350 lines)
EliteRestaurantPro/Views/ReportsView.xaml.cs (11 lines)
EliteRestaurantPro/Views/RoleSelectionView.xaml (168 lines)
EliteRestaurantPro/Views/RoleSelectionView.xaml.cs (8 lines)
EliteRestaurantPro/Views/TablesView.xaml (208 lines)
EliteRestaurantPro/Views/TablesView.xaml.cs (11 lines)

9) Reality check and next steps

- The architecture is coherent and practical for a local-first restaurant management app.
- Strongest immediate improvements: real authentication, tests, and smaller service-oriented ViewModels.
- Schema management currently happens at runtime; long-term maintainability improves with explicit migrations.
- If project grows into multi-user environment, plan API/server sync strategy and conflict resolution.
- If you want, next version can include clickable table of contents and per-file mini summaries.
- Another upgrade can add sequence diagrams for Order->Inventory->Money and Attendance->Salary flows.
- You can regenerate this guide anytime by running this script after code changes.